

An Introduction to Building Spring Boot Applications



```
@GetMapping("/posts/filter")
public ResponseEntity<List<Post>> getPostByFilters(@RequestParam String tag, @
RequestParam String type) {
    return ResponseEntity.ok(service.getByTagsAndType(tag, type));
}

@PostMapping("/posts")
public ResponseEntity<Data> addPost(@RequestBody post) {
    return ResponseEntity.ok(service.addPostToTimeline(post));
}

@GetMapping("/posts/filter")
public ResponseEntity<List<Post>> getPostByFilters(@RequestParam
String tag, @RequestParam String type) {
    return ResponseEntity.ok(service.getByTagsAndType(tag, type));
}
```

Spring is one of the most popular frameworks for Java based systems, and with good reason. The Spring environment is vast, and thousands of man-hours have gone into making it a success. Spring allows developers to create modern enterprise-grade apps without having to worry about underlying systems, allowing them to focus on the application's business logic. It can be leveraged to build very powerful applications without writing too much boilerplate.

Spring has strong opinions about how components should function. As part of the Spring Core module, it provides an application context as well as a bean factory for creating Spring-managed objects known as 'Spring Beans'.

To allow inversion of control, a dependency injection container offered by Spring is used to handle dependencies. The process of injecting objects into Spring modules is known as 'Autowiring', and it is accomplished by tagging dependencies with the '@Autowired' annotation. Annotations are used a lot in Spring to offer settings. Spring Boot services, for example, can be annotated with '@

Service', while data access repositories can be annotated with '@Repository'.

Spring Boot allows you to quickly create Spring based applications with little configuration and boilerplate code. It uses the 'just works' technique, which allows a Spring Boot application to be initialised and deployed in under a minute with only a few clicks.

This has allowed enterprises to build a large number of little apps known as microservices on top of Spring Boot in order to segregate concerns and perform them effectively, while connecting them together brings the complete application to the user.